

第2章介绍的内置类型是由C++语言直接定义的。这些类型，比如数字和字符，体现了大多数计算机硬件本身具备的能力。标准库定义了另外一组具有更高级性质的类型，它们尚未直接实现到计算机硬件中。

本章将介绍两种最重要的标准库类型：`string` 和 `vector`。`string` 表示可变长的字符序列，`vector` 存放的是某种给定类型对象的可变长序列。本章还将介绍内置数组类型，和其他内置类型一样，数组的实现与硬件密切相关。因此相较于标准库类型 `string` 和 `vector`，数组在灵活性上稍显不足。

在开始介绍标准库类型之前，先来学习一种访问库中名字的简单方法。



3.1 命名空间的 using 声明

目前为止，我们用到的库基本上都属于命名空间 `std`，而程序也显式地将这一点标示了出来。例如，`std::cin` 表示从标准输入中读取内容。此处使用作用域操作符 (`::`) (参见 1.2 节，第 7 页) 的含义是：编译器应从操作符左侧名字所示的作用域中寻找右侧那个名字。因此，`std::cin` 的意思就是要使用命名空间 `std` 中的名字 `cin`。

上面的方法显得比较烦琐，然而幸运的是，通过更简单的途径也能使用到命名空间中的成员。本节将学习其中一种最安全的方法，也就是使用 **using 声明** (using declaration)，18.2.2 节 (第 702 页) 会介绍另一种方法。

有了 `using` 声明就无须专门的前缀 (形如命名空间 `::`) 也能使用所需的名字了。`using` 声明具有如下的形式：

```
using namespace::name;
```

一旦声明了上述语句，就可以直接访问命名空间中的名字：

```
#include <iostream>
// using 声明，当我们使用名字 cin 时，从命名空间 std 中获取它
using std::cin;

int main()
{
    int i;
    cin >> i;           // 正确：cin 和 std::cin 含义相同
    cout << i;           // 错误：没有对应的 using 声明，必须使用完整的名字
    std::cout << i;      // 正确：显式地从 std 中使用 cout
    return 0;
}
```

每个名字都需要独立的 using 声明

按照规定，每个 `using` 声明引入命名空间中的一个成员。例如，可以把要用到的标准库中的名字都以 `using` 声明的形式表示出来，重写 1.2 节 (第 5 页) 的程序如下：

83

```
#include <iostream>
// 通过下列 using 声明，我们可以使用标准库中的名字
using std::cin;
using std::cout; using std::endl;
int main()
{
    cout << "Enter two numbers:" << endl;
```

```
int v1, v2;
cin >> v1 >> v2;
cout << "The sum of " << v1 << " and " << v2
    << " is " << v1 + v2 << endl;
return 0;
}
```

在上述程序中，一开始就有对 `cin`、`cout` 和 `endl` 的 `using` 声明，这意味着我们不用再添加 `std::` 形式的前缀就能直接使用它们。C++ 语言的形式比较自由，因此既可以一行只放一条 `using` 声明语句，也可以一行放上多条。不过要注意，用到的每个名字都必须有自己的声明语句，而且每句话都得以分号结束。

头文件不应包含 using 声明

位于头文件的代码（参见 2.6.3 节，第 67 页）一般来说不应该使用 `using` 声明。这是因为头文件的内容会拷贝到所有引用它的文件中，如果头文件里有某个 `using` 声明，那么每个使用了该头文件的文件就都会有这个声明。对于某些程序来说，由于不经意间包含了一些名字，反而可能产生始料未及的名字冲突。

一点注意事项

经本节所述，后面的所有例子将假设，但凡用到的标准库中的名字都已经使用 `using` 语句声明过了。例如，我们将在代码中直接使用 `cin`，而不再使用 `std::cin`。

为了让书中的代码尽量简洁，今后将不会再把所有 `using` 声明和 `#include` 指令一一标出。附录 A 中的表 A.1（第 766 页）列出了本书涉及的所有标准库中的名字及对应的头文件。



读者请注意：在编译及运行本书的示例前请为代码添加必要的 `#include` 指令和 `using` 声明。

3.1 节练习

练习 3.1：使用恰当的 `using` 声明重做 1.4.1 节（第 11 页）和 2.6.2 节（第 67 页）的练习。

3.2 标准库类型 string



标准库类型 **string** 表示可变长的字符序列，使用 `string` 类型必须首先包含 `string` 头文件。作为标准库的一部分，`string` 定义在命名空间 `std` 中。接下来的示例都假定已包含了下述代码：

```
#include <string>
using std::string;
```

本节描述最常用的 `string` 操作，9.5 节（第 320 页）还将介绍另外一些。



C++ 标准一方面对库类型所提供的操作做了详细规定，另一方面也对库的实现者做出一些性能上的需求。因此，标准库类型对于一般应用场合来说有足够的效率。